C and Software Tools
Exam 1

Name _____

Unity/EOS ID _____

- This test is worth 100 points. There are actually 106 points available. If you score more than 100, you've earned yourself some extra credit.
- This test consists of 9 pages. Make sure your copy has them all.
- Use the space provided for your answer to each problem. If you run out of room, you can continue on to the back of the next page (or the previous page). Be sure to indicate this at the bottom of the answer space, so that parts of your answer aren't overlooked.
- Read each problem carefully. You don't want to lose points just because you overlooked part of a question.
- In general, you don't need to comment the code you write or follow our course style guidelines, unless the question specifically asks for it.
- In general, you don't need to handle errors (e.g., buffer overflow or being unable to open a file), unless the question specifically asks for it.
- Where it matters, you should base your answers on our common platform.
- This exam is closed book and closed notes. If you've prepared a handwritten 3×5 card of notes, you may use it during the exam, but your name must be clearly printed on your notecard and you must turn it in at the end of the exam. You may **not** use a calculator or any other device in completing this exam. Your work must be individual, and cheating will be punished according to university academic integrity code.

On my honor: I affirm that I have not and will not use any sources of assistance (notes, another student's test, etc.) that are not explicitly authorized during this exam. I am the student whose name appears on this exam.

Signed: _____

Date _____

1. **Expression Typing**, Assume the following variables have been declared (I've tried to make their types easy to remember):

```
unsigned char uc;
short s;
int i;
unsigned int ui;
long l;
unsigned long ul;
long long ll;
unsigned long long ull;
float f;
double d;
```

In each of the following expressions, one operator has a box drawn around it. Consider any implicit conversions the compiler will introduce in this expression and report to the right the type of operands this operator will have when it's evaluated (i.e., if its operands are converted, what type will they be converted to?). Where it matters, you should base your answers on our common platform. (1 pts each)

Expression	**type**
`i` **`+`** `ui`	unsigned int
`d + s` **`+`** `i`	double
`ll` **`+`** `f`	float
`i / ui` **`-`** `l / ul`	unsigned long
`ll + l + i` **`+`** `s + uc`	long long
`ll * i` **`+`** `ul`	unsigned long long

2. **Program Fragments**, For each of the following, give a fragment of C code that accomplishes the requested task. Where it matters, you can assume that the code you are writing will be executed on the common platform in order; the code for a later question can use variables declared in an earlier question. (2 pts each)

   (a) Declare a character `ch` and initialize it to hold the newline character.

   ```
   char ch = '\n';
   ```

   (b) Declare a variable named `list` as an one-dimensional array of int. Use initialization syntax to set `list` to contain the sequence 1, 2, 4, 8, 16, 32, 64.

   ```
   int list[] = { 1, 2, 4, 8, 16, 32, 64 };
   ```

   (c) Declare a variable `s` and, using the appropriate operator, initialize `s` to the number of bytes used to store `list`.

   ```
   size_t s = sizeof( list );
   ```

   (d) Give a prototype for a function `f()` that takes a pointer to an int as a parameter and returns nothing. Specify that `f()` won't change the contents of the memory pointed two by its parameter.

   ```
   void f( int const *p );
   ```

   (e) Replace the initial values in the `list` array with values read from standard input. You don't have to prompt the user; just assume they'll know what to enter.

   ```
   scanf( "%d", &list[ 0 ] );
   ```

   (f) Declare a variable `p` and set it to contain the address of variable `ch` (define above).

   ```
   char *p = &ch;
   ```

   (g) Using `p`, change the value of `ch` to a space.

   ```
   *p = ' ';
   ```

   (h) Declare an unsigned int variable `x` and initialize it with the binary value (remember, C99 doesn't let you give values in binary, but there are other easy ways to initialize this variable): 11001111001110111000011000001110.

   ```
   unsigned int x = 0xCF3B860E; // Either in hex
   unsigned int x = 031716703016; // or in octal
   ```

3. **Strings,** Give a code fragment to create an array named `str` containing the string "Hello World". (2 pts)

```
char str[] = "Hello World";
```

   What value would the last element of the array `str` contain? (1 pts)

   A null terminator, character `'\0'`.

4. **Arrays and Formatted Output,** Pretend a one-dimensional array of doubles named `vlist` has been declared in main() and initialized to contain a sequence of values. Give a fragment of code that main() could use to print out this sequence of values, one value per line, rounded to 4 fractional digits. I'm not telling you how many values `vlist` contains; your code should be able to determine this. (5 pts)

```
for ( int i = 0; i < sizeof( vlist ) / sizeof( vlist[ 0 ] ); i++ )
  printf( "%.4f\n", vlist[ i ] );
```

5. **Compilation,** So far, we've only used the preprocessor for two things. What are they? (4 pts)

   We use it to define preprocessor macros used as constants.

   We also use it to include header files.

6. **Type Ranges,** You want to set an unsigned int, `v` to the largest possible value it can hold. Breifly describe how you could do that. (You don't have to show code, but you can if you want.) (2 pts)

   You can include limits.h and set v to UINT_MAX.

   After you execute `v++`, what value would `v` contain? (1 pts)

   This will overflow, and v will will end up as zero.

7. **Character Input,** In the space below, write a program fragment that reads from standard input one character at a time until the end-of-file is reached. Count the total number of capital letters read, and print this to standard output. For example, on input "aBC 123 XyZ", your code should print 4. (7 pts)

```
int ch;
int count = 0;
while ( ( ch = getchar() ) != EOF ) {
  if ( ch >= 'A' && ch <= 'Z' )
    count++;
}
printf( "%d\n", count );
```

8. **Program Execution,** Pretend you have a program named "sweep" in the current directory. Give a shell command that could be used on the common platform to run this program, with input being read from the file "in.txt" rather than standard input. Then, give a command to report the program's exit status once the program is done. (4 pts)

```
# to run the program as requested
./sweep < in.txt

# to examine the exit status
echo $?
```

9. **Pass By Reference,** In the space below, give a function named `split()`. This function will take two double parameters passed by address, `a` and `b`, and an integer percentage, `p`, passed by value. The job of the function will be to decrease the value of `a` by `p` percent and set `b` to `p` percent of `a`. For example, if called with values 300.0, 50.0 and 10, the function will change `a` to 270.0 (300.0 decreased by 10 percent) and set `b` to 30.0 (10 percent of 300.0) (6 pts)

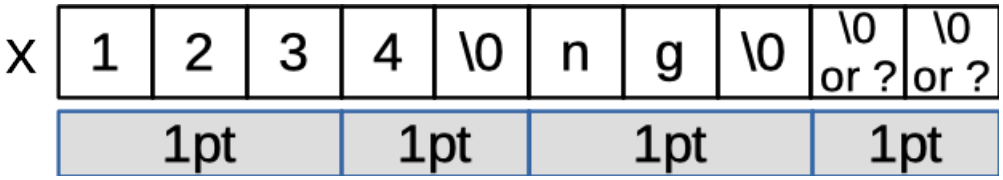
```
void split( double *a, double *b, int p ) {
  *b = *a * ( p / 100.0 );
  *a -= *b;
}
```

10. **Binary Arithmetic,** Consider the following 8-bit binary addition problem. Fill in the result of this addition below. (3 pts)

$$\begin{array}{rr} & 10001101 \\ + & 01111001 \\ \hline & 00000110 \end{array}$$

11. Consider the following fragment of code. Show what the array `x` will contain after this code executes. Put a question mark in any array elements where it's impossible to know what that element will contain. (4 pts)

```
char x[ 10 ] = "testing";
strcpy( x, "123" );
strcat( x, "4" );
```



12. **Fill-in the Blank,** For each of the following, write in the best word(s), term(s) or bits of C syntax to complete the statement. (2 pts each)

    (a) When we compile a C program with gcc, we normally give the compiler two command-line options, the `-Wall` option enables additional warnings. The other option says that we would like to use the **C99** standard of the C language.

    (b) As a unary prefix operator, `*` is called the **pointer dereference** operator.

    (c) Typically, setting a pointer to the value **NULL** indicates that it doesn't point to anything.

    (d) The **stdbool** header lets us use the name, `bool`, for the boolean type, instead of `_Bool`.

    (e) An auto variable is allocated in a region of the program's memory called the **stack**.

    (f) During execution of a Java program, the Java virtual machine will load additional components as they are needed. In general, C programs don't work like this. The **linker** is responsible for combining all the needed components into an executable before execution.

    (g) A(n) **static local** variable has a scope of just one function, but its lifetime is the entire execution of the program.

    (h) Consider the following code fragment. **sign extension** occurs when you assign from x to y. It involves copying the high-order bit of x into the additional bits to the left in y.

```
int y;
short x = -50;
y = x;
```

13. **Multiple Choice,** Mark the best answer for each of the following questions. (3 pts each)

(a) Consider the binary addition problem in Question 10. As an 8-bit addition, does this addition represent an overflow?

- ◯ Yes. It overflows.
- ◯ No. It doesnt overflow.
- ☑ It depends on whether the addition is signed or unsigned.
- ◯ It depends whether these numbers were given in decimal, octal or hexidecimal.

(b) We say a program terminates successfully if it terminates with an exit status of:

- ☑ zero ◯ anything other than zero
- ◯ one ◯ anything other than one

(c) A function prototype is:

- ☑ a declaration but not a definition
- ◯ a definition but not a declaration
- ◯ both a definition and a declaration
- ◯ neither a definition nor a declaration

(d) On the common platform, which of the following is declaration of a 25-element array of pointers to 16-bit signed integers?

- ◯ `int *(a[ 25 ]);` ◯ `(char *)a[ 25 ];`
- ☑ `short *a[ 25 ];` ◯ `int (*a)[ 25 ];`

(e) On the common platform, how much memory will it take to store the array described in Question 13d?

- ◯ 25 bytes ◯ 50 bytes
- ◯ 100 bytes ☑ 200 bytes

(f) In general, scanf() skips whitespace when it's trying to match a format specifier. Which format specifier doesn't skip whitespace?

◯ `%d` ◯ `%ld` ◯ `%lf` ☑ `%c`

(g) If you write to elements outside the bounds of an array, what will happen?

- ◯ Your program will crash.
- ◯ An exception will be thrown.
- ◯ The compiler won't let you do this.
- ☑ Behavior is undefined.

(h) Pretend int variables a, b, c and d contain 3, 4, 5 and 6 respectively. What will this expression evaluate to: `a < b ? ( a, c ) : ( b, d )`

◯ 3 ◯ 4 ☑ 5 ◯ 6

(i) How many tokens would the C compiler find in parsing the following input (I know this wouldn't compile, but we can still think about the tokens it contains):

```
a2c+++1c3===
```

◯ 3 ◯ 4 ◯ 5 ◯ 6 ☑ 7 ◯ 8 ◯ 9

(j) The scanf() function returns:

- ◯ the number of characters successfully read
- ◯ true if all conversion specifiers are successfully matched
- ☑ the number of conversion specifiers successfully matched
- ◯ nothing